Contest Module: Design Patterns, SOLID & Performance Review

Where to apply which pattern, why, and in what order, so the contest system scales, stays maintainable and fails safely

Project	your_capture_award_api (Node.js / Express / TypeScript / Prisma on MongoDB)
Scope	src/app/modules/Contest/** plus the modules it is wired to: Vote, PhotographyOfTheYear, Agenda jobs, Team, Payment, Notification, Prize, RecurringContest. The performance section covers the whole backend
Snapshot	Branch **poty**, commit d70a7d0 plus uncommitted work in the tree on 31 Oct 2026
Method	Every Contest source file was read line by line. Cross-module call sites were traced with grep. Performance figures come from running the real service code on a seeded local test database. Line numbers refer to that snapshot.
Audience	Backend developers who maintain or refactor the contest system

How to read this report. Section 1 is a two-page summary. Sections 3 and 4 diagnose the code. Section 5 is the core: 14 pattern recommendations, each with the exact location, the problem as it is today, the pattern, why it fits, a TypeScript sketch written in the codebase's own functional style, and the effort. Section 6 lists reliability defects found during the analysis. Section 7 is the measured performance analysis. Section 8 is the step-by-step migration plan.

Contents

1. Executive summary 4

Top recommendations by value . . . 4

Defects found during the analysis . . . 4

Performance findings (measured) . . . 5

2. Scope, method and measurements 6

Files analysed . . . 6

Signals that measure the coupling . . . 6

Dependency graph (problem edges) . . . 6

3. Patterns already in the codebase 8

4. SOLID assessment of the Contest module 9

Duplication inventory (DRY violations that patterns will remove) . . . 9

5. Pattern recommendations (detailed) 11

P1. Decompose the god service behind a Facade . . . 11

P2. Repositories + Unit of Work + guarded-mutation template . . . 12

P3. Contest lifecycle as an explicit state machine . . . 13

P4. Contest-type policies instead of isPhotographyOfYear branches . . . 14

P5. One command runner for paid in-contest actions . . . 15

P6. Rule engine as a handler registry with one rule-set load . . . 16

P7. A single vote-tally function . . . 17

P8. Domain events + transactional outbox for side effects . . . 17

P9. Contest blueprint factory (replace the unused builders) . . . 18

P10. Banner and storage behind an adapter . . . 19

P11. Caching as a decorator around read loaders . . . 20

P12. Contest list queries as specifications . . . 20

P13. Presenters / mappers for response shapes . . . 21

P14. Thin controllers with validated request DTOs . . . 21

6. Reliability findings 23

Unused code to delete before refactoring . . . 24

7. Performance analysis 25

How this was measured . . . 25

Results at a glance . . . 25

PF1. Votes fail when several arrive at once (Critical) . . . 26

PF2. 'Votes received by a user' scans every vote in the database (Critical) . . . 26

PF3. The ranking is rebuilt constantly, and one index is missing (Critical) . . . 27

PF4. One vote starts 127 database operations in the background (Critical) . . . 27

PF5. Real-time events do not cross workers (High) . . . 28
PF6. Missing indexes (High) . . . 28
PF7. The voting feed is slow for every page (High) . . . 28
PF8. Every authenticated request reloads the full user (High) . . . 28
PF9. Admin dashboard loads whole tables into memory (High) . . . 29
Other findings (Medium) . . . 29
Performance action plan . . . 29
8. Implementation roadmap **31**
Testing strategy that makes this safe . . . 31
Target module layout . . . 31
9. Guardrails: when not to apply a pattern **32**
Appendix A. Pattern to location matrix **32**

1. Executive summary

The Contest module is about **8,000 lines**. **3,272** of them are in one file, `contest.service.ts`, which exports **40 operations** covering at least ten separate responsibilities: contest authoring, recurring materialization, entry and payment, photo submission, photo trading, boosts, listing and detail read models, leaderboards, moderation, and exposure decay. The supporting subsystems (rules, ranking, finalization, cache) are already in separate files and are generally well built.

The reliability foundation is better than average. The code already uses lease-based claims for finalization, idempotency keys for recurring occurrences and award grants, guarded conditional writes inside transactions, a versioned Redis cache that fails open, single-flight memoization of ranking builds, and a write-conflict retry helper. These should be kept and generalised, not replaced. The one exception is the write guard on votes, which causes failures under load (PF1).

Maintainability and extensibility are the bottleneck. The same logic is copied in several places: contest row construction (3 copies), rule-config insertion (4), the active-contest guard (5), store-item spending (3), post-commit cache invalidation (7 or more), vote counting (4 implementations that do not fully agree), and the `isPhotographyOfYear` branch (28 references inside the module). Adding a new contest type, a new rule, or a new paid action means editing many unrelated functions, and any one of those edits can introduce drift.

Top recommendations by value

Ref	Recommendation	Pattern(s)	Main benefit
P1	Split contest.service.ts into cohesive application services behind an unchanged facade	Facade, SRP decomposition	Maintainability; parallel work; smaller blast radius
P3	Make contest status changes go through one transition table	State (state machine)	Reliability; no illegal or racing transitions
P4	Move every Regular vs. POTY difference into a policy object	Strategy	Open/Closed; new contest types without editing core flows
P7	Use one pure vote-tally function everywhere	Strategy / single source of truth	Consistent numbers on every screen
P5	Spend boosts, trades and charges through one guarded command runner	Command + Template Method	Fixes a real race; uniform auditing
P6	Use a rule-handler registry with one rule-set load per request	Registry + Strategy + Chain of Responsibility	New rule = one file; fewer DB reads
P8	Publish domain events with an outbox for side effects	Observer + Transactional Outbox	Decoupling; removes the Contest/Team import cycle
P9	Build contests from a blueprint shared by create and materialize	Factory + Builder (fixed)	Removes three duplicated insert paths

Defects found during the analysis

The analysis found 2 high-severity and 7 medium-severity reliability issues (Section 6). The two that matter most:

- **Promote can overspend boosts.** The balance is checked outside the transaction and decremented unconditionally inside it (contest.service.ts:2554-2577), so concurrent requests can push `boost` below zero.
- **Admin photo removal can race finalization.** The status check and the update are separate, non-transactional steps (contest.service.ts:1676-1684), even though the code itself says the photo set must be frozen once finalization starts (lines 53-55).

Section 6 also lists two numeric inconsistencies that need product confirmation (vote counts on My Contests vs. the leaderboard, and reward labels in finalization notifications), plus dead or latent-buggy code that should be deleted before refactoring.

Performance findings (measured)

Running the real code on 1 million test votes (Section 7) found four critical problems. The first is the most urgent because users see failed votes:

- **Votes fail under concurrency.** With 4 votes at once on one contest, 75% failed; with 16, 93% failed. Every vote writes the same contest document, so MongoDB aborts overlapping transactions, and they are not retried.
- **Vote level checks scan every vote.** One user's received-vote total took 146 seconds. A two-step query returned the same totals in 28 ms.
- **The ranking is rebuilt constantly.** One missing index accounts for most of a 22.6-second build (3.6 s with the index), and each vote discards the cached ranking.
- **One vote starts 139 seconds of background work** (127 database operations, 4.29 million documents examined), with no limit on how much can pile up.

These figures depend on data size, so check your real vote count first. The failure pattern in the first item does not depend on data size.

Suggested sequencing: start with step 1 of the performance plan in Section 7, then fix the confirmed defects and add characterization tests first (about 1 week), then extract shared primitives (state machine, guarded mutation, vote tally, consumables), then split the service, then introduce strategies and events. Every step keeps the public `contestService` API and the HTTP contract unchanged, so the work can ship incrementally.

2. Scope, method and measurements

Files analysed

File (src/app/modules/Contest/...)	LOC	Responsibility today
contest.service.ts	3,272	Almost everything: create/update/delete, recurring materialization, join + paid join, uploads, trade, promote, charge, exposure decay, all list/detail read models, leaderboards, moderation + email
ContestFinalization/contestFinalization.service.ts	701	Lease claim, freeze, ranking snapshot, award + level grant ledger, reward payout, notifications, award selection
contest.controller.ts	493	HTTP handlers; manual query parsing
ContestRules/contestRule.engine.ts	405	Join, upload, format, duplicate and voting validation
ContestRanking/contestRanking.service.ts	392	Streaming vote scan, scoring, tie-breaks, in-process memo, snapshot persistence
ContestRules/contestRule.definitions.ts + validation + service	791	Rule catalogue, admin field metadata, zod schemas, normalisation, summaries
contest.cache.ts	232	Versioned Redis cache-aside with metrics; fails open
contest.validation.ts, contestCategories.ts, contestCategory.service.ts	419	Request schemas, multi-category normalisation, category registry
classes/* (builders, factory)	100	Builder + Factory. **Unused**, and the date check is inverted
v2/*, ContestPhoto/contestPhoto.service.ts, contest.enum.ts, contest.controller.spec.ts	4	Empty placeholders

Collaborators read: Vote/vote.service.ts (454), PhotographyOfTheYear/* (867), Agenda/jobs.ts (233), Team/team.service.ts (imports only, 3,207), Payment/* (contest entry path), Notification/notificationOrchestrator.ts, Prize/prize.service.ts (exports), event/*, Uploader/classes/*.

Signals that measure the coupling

Signal	Value	What it tells us
Exported operations on contestService	40	Interface Segregation pressure: Agenda uses 3, Dashboard 1, Team 1, POTY 2, yet each imports the whole thing
prisma.$transaction blocks in Contest module	19	Each one hand-rolls guards and post-commit work
ApiError throws in Contest module	109	Validation and HTTP concerns are mixed into domain code
isPhotographyOfYear references in Contest module	28	Type-specific behaviour spread across flows (Open/Closed violation)
Manual cache/ranking/profile invalidation calls (all modules)	62	Side effects are coupled to every writer
Copies of the notDeleted filter	5	contest.service.ts:86, contestLifecycle.ts:6, Agenda/jobs.ts:8, photographyOfTheYear.service.ts (x2)
Rule summary formatter copies	2	contest.service.ts:1281-1311 and contestRules.service.ts:170-200 are the same switch
Service-level automated tests	0	The 4 contest test files cover pure helpers only (validation, categories, photo input, submission format)

Dependency graph (problem edges)



```
Vote.service ---> contestRuleEngine, contestRanking, contestProgress, Level, Notification
contestService -> voteService (counts)   and   voteService -> contestRanking (counts)
```

The two cycles are currently broken at runtime with a dynamic import and by careful ordering. That works, but it is fragile, and it is exactly the kind of coupling that domain events (recommendation P8) remove.

3. Patterns already in the codebase

Before adding patterns it helps to know which ones already exist, which work, and which should be fixed or removed. Several reliability patterns here are production-grade and should become the template for the rest of the module.

Pattern (where)	Status	Verdict
Lease / claim with stale takeover (claimFinalization, finalization.service.ts:117-148; occurrence claim, contest.service.ts:500-524; processGrant 303-318)	Working	Excellent. Generalise into one `claimLease()` helper; it is copied three times
Idempotency keys / ledger (grantKey, occurrenceKey, ContestRewardTransaction.grantId unique)	Working	Excellent. Keep as is. It is what makes finalization safely retryable
Guarded conditional write (updateMany with a where-guard, then check count === 1)	Working, but copied	Correct for conditional updates, and for rare actions such as trades. **Corrected after measurement:** using it to serialize votes on the contest document causes PF1, so do not extend it to high-frequency writes. Extract into a Template Method (P2/P5)
Cache-aside with version keys (contest.cache.ts)	Working	Very good design: race-free invalidation, fails open, metrics. Keep it; move the call sites out of business functions (P11)
Single-flight memoization (buildContestRanking, ranking.service.ts:280-312)	Working	Good. It is per-process, so see R13 on multi-instance deploys
Retry with backoff (shared/transactionRetry)	Working	Good. Use it from the Unit-of-Work helper
Registry of rule definitions (contestRule.definitions.ts)	Partial	Data-driven catalogue, but behaviour per rule still lives in switch statements spread across 4 files (P6)
Builder + Simple Factory (classes/AbstractContestBuilder etc.)	Unused, buggy	The date guard is inverted (throws when end >= start), recurrence() does not return this, and the any-typed state breaks LSP. Replace with the Blueprint factory (P9) or delete
Factory + Registry + plugin loader (Payment/paymentFactory.ts, paymentRegistry.ts, providerLoader.ts)	Two mechanisms	Pick one (the Registry). Two ways to get a provider will drift
Abstract provider (Uploader/classes/*)	Thin	Fine as an Adapter seam for storage
Event emitter (event/eventEmitter.ts, events.constant.ts)	Unused	Untyped and in-memory; no publishers. Revive as a typed bus plus outbox (P8)
Orchestrator facade (Notification/notificationOrchestrator.ts)	Working	A good Facade over DB, socket and queue. Contest code should talk only to it (no inline e-mail HTML)

4. SOLID assessment of the Contest module

S - Single Responsibility [Poor]

Evidence. contest.service.ts mixes ten responsibilities. Single functions combine validation, file I/O (banner upload/discard), pricing, persistence, cache invalidation, team matchmaking and e-mail. Examples: uploadPhotoToContest (2244-2410) does rule validation, payment gating, S3 upload with rollback, participant creation, fee charging, slot creation, category tagging, cache invalidation and team-queue notification. adminDeleteContestPhoto (1664-1716) builds HTML e-mail inline.

Direction. Split by use case (P1). Push side effects to events (P8). Move presentation to presenters (P13).

O - Open/Closed [Poor]

Evidence. Adding a contest type means editing createContest, updateContest, joinContest, uploadPhotoToContest, vote.service.addOneVote and finalizeContest (28 isPhotographyOfYear checks). Adding a rule means editing definitions (2 switches), the engine, the summary formatter (2 copies) and validation. Status semantics are re-declared as arrays in at least 6 places.

Direction. Strategy per contest type (P4), rule-handler registry (P6), state machine (P3).

L - Liskov Substitution [Weak]

Evidence. The builder hierarchy is not substitutable: build() returns Contest | RecurringContest from an any-typed bag, and RecurringContestBuilder.recurrence() breaks chaining. Structural tx types (ContestEntryFeeTx, ContestPhotoCategoryTx) are a good LSP-friendly start.

Direction. Replace builders with a typed blueprint (P9). Keep structural ports.

I - Interface Segregation [Poor]

Evidence. One 40-method object is imported by Agenda, Team, POTY, Dashboard and RecurringContest. Boolean flag parameters (allowPhotographyOfYear on updateContest, joinContest, uploadPhotoToContest) create hidden alternative interfaces inside one function.

Direction. Narrow service modules (P1). Replace flag arguments with a policy parameter (P4).

D - Dependency Inversion [Weak]

Evidence. Every function imports the prisma singleton, redis caches, agenda, fileUploader and sendMail directly, so nothing can be unit-tested without a database. Exceptions worth copying: registerContestCategoryNames(db, ...) and chargeContestEntryFee(tx, ...) already take their dependency as a structural parameter.

Direction. Repositories + Unit of Work (P2), ports for clock, scheduler, storage and mailer. Generalise the existing structural-tx approach.

Duplication inventory (DRY violations that patterns will remove)

Duplicated logic	Locations	Resolved by
Contest row + rules + awards + level-award insertion	createContest 396-449; createRecurringContest 686-741; materializeRecurringOccurrence 542-612; updateContest 852-913	P9 Blueprint factory
Banner resolution (user photo vs. upload vs. discard)	372-378; 705-714; 817-832	P10 BannerResolver
Active-contest guard inside a transaction	1642-1648; 2359-2362; 2560-2566; 2700-2706; vote.service addOneVote	P2 withActiveContest()
Store item spending (boost/swap/key/coins)	2554-2577 promote; 2675-2731 trade; 2867-2882 charge; 203-230 entry fee	P5 consume() + action runner

Duplicated logic	Locations	Resolved by
Post-commit invalidation sequences	1030-1033; 1108-1111; 2340-2344; 2585-2588; 2783-2785; 2889; 1656-1657	P8 events
Vote counting formula	ranking.service 164-199; vote.service getVoteCount + getVoteCountsForContestPhotos; contest.service 1603-1608 and 2977-2981	P7 VoteTally
Paginated contest list (where + order + count + enrich)	getAllContests 1196; getPublicContests 1238; getContestsByStatus 1513	P13 Query specs
Uploads view mapping	getContestUploads 2101-2142 vs. getCompletedContestUploads 2064-2097 (dead)	P14 Presenters
Status set definitions	48-64, 778-784; jobs.ts 33, 86, 146-157; finalization 484-491	P3 State machine

5. Pattern recommendations (detailed)

Every sketch below follows the codebase's existing style: modules exporting object literals of functions, Prisma types, ApiError. Patterns do not require classes, and in this codebase a function plus a typed object is usually the clearer form. Effort figures are rough estimates for one developer familiar with the code and include tests.

P1. Decompose the god service behind a Facade

Pattern	Facade + application services (one module per use-case cluster)
Where	contest.service.ts (whole file); contest.controller.ts; every external importer of contestService
SOLID	S (one reason to change per module), I (narrow imports)
Effort / risk	4-6 days | Low: mechanical moves behind an unchanged facade

Problem today. 3,272 lines and 40 exports in one module. Any change, even to leaderboards, touches a file that also holds money-moving code (entry fees, store spending). Merge conflicts and review load concentrate in one place. Importers such as Agenda and Team depend on all 40 operations and create the cycles shown in Section 2.

Why this pattern. A Facade keeps the existing `contestService` object and HTTP contract, so callers do not change, while the implementation is split into cohesive modules that can be tested and owned separately. This is the enabling step for every other recommendation, and it is low risk because it is mostly moving code.

```
// Contest/
//   authoring/contestAuthoring.service.ts   create, update, softDelete, createOptions,
    bannerCandidates
//   recurring/recurringMaterializer.ts      materializeRecurringOccurrence, createRecurringContest
//   entry/contestEntry.service.ts           join, completePaidJoin, submitPhotos, withdrawPhoto
//   actions/contestPhotoActions.ts          promote, trade, charge (+ tradeableHistory)
//   exposure/exposure.service.ts            decayExposureMeters, voting-feed weighting
//   read/contestQuery.service.ts            lists, details, my-active, my-completed
//   read/contestLeaderboard.service.ts      rank-photos, rank-photographer, ranking
//   moderation/contestModeration.service.ts adminDeleteContestPhoto, participants
//   contest.service.ts                      FACADE ONLY (re-exports, no logic)

import { contestAuthoring } from "./authoring/contestAuthoring.service";
import { contestEntry } from "./entry/contestEntry.service";
// ...
export const contestService = {
    ...contestAuthoring,
    ...contestEntry,
    ...contestPhotoActions,
    ...contestQuery,
    ...contestLeaderboard,
    ...contestModeration,
    materializeRecurringOccurrence: recurringMaterializer.materialize,
    identifyWinner: contestFinalizationService.finalizeContest,
    decayExposureMeters: exposureService.decay,
};

// New code (Agenda, Team, POTY) imports the narrow module directly:
import { recurringMaterializer } from "../Contest/recurring/recurringMaterializer";
```

P2. Repositories + Unit of Work + guarded-mutation template

Pattern	Repository, Unit of Work, Template Method
Where	All 19 prisma.$transaction blocks; the guard at contest.service.ts:1642, 2359, 2560, 2700 and vote.service addOneVote; the transaction typings at contest.service.ts:66-81
SOLID	D (depend on repo ports), S (transaction plumbing separated from rules), O (new use cases reuse the skeleton)
Effort / risk	5-7 days (incremental, per use case) | Medium: touches transactional code. Migrate one use case per PR with tests

Problem today. Each mutating use case repeats the same skeleton: open a transaction, re-check that the contest is active with a harmless updatedAt write, perform the change, then run three or four post-commit invalidations. The skeleton is correct but copied, and the copies have already diverged: adminDeleteContestPhoto skips it entirely (R2). Direct prisma imports also make service logic untestable without MongoDB.

Why this pattern. A Template Method fixes the invariant steps (lock, then mutate, then run after-commit hooks) and lets each use case supply only the variable part. Repositories turn Prisma calls into named intentions (`lockActive`, `findSlotForOwner`), which documents the domain and lets tests pass in-memory fakes. The codebase already does this in miniature (ContestEntryFeeTx, ContestCategoryDb), so this generalises a local convention.

Correction after performance testing. The sketch below keeps the existing updatedAt write in lockActive. That is acceptable for trades, promotes and uploads, but **votes must not use it**: it makes all votes in a contest conflict on one document (see PF1). Use a read-only check for votes.

```
// shared/unitOfWork.ts
export type ContestRepos = ReturnType<typeof createContestRepos>;

export const createContestRepos = (db: PrismaTx) => ({
    contests: {
        // The serialization guard used by votes, uploads, trades and promotes.
        lockActive: async (contestId: string) =>
            (await db.contest.updateMany({
                where: { id: contestId, ...activeContestWhere() },
                data: { updatedAt: new Date() },
            })).count === 1,
    },
    slots: {
        findLiveForOwner: (slotId: string, contestId: string, userId: string) =>
            db.contestPhoto.findFirst({ where: { id: slotId, contestId, photoId: { not: null },
                participant: { userId, status: ContestParticipantStatus.ACTIVE } } }),
    },
    store: { consume: (userId: string, item: StoreItem, amount = 1) => consume(db, userId, item,
        amount) },
});

type AfterCommit = () => Promise<unknown>;

// Template Method: invariant skeleton, variable "work".
export const withActiveContest = async <T>(
    contestId: string,
    work: (repos: ContestRepos, afterCommit: (fn: AfterCommit) => void) => Promise<T>,
    options = { maxWait: 5000, timeout: 15000 },
) => {
    const hooks: AfterCommit[] = [];
    const result = await runWithWriteConflictRetry(() => prisma.$transaction(async tx => {
        const repos = createContestRepos(tx);
        if (!(await repos.contests.lockActive(contestId))) {
            throw new ApiError(httpstatus.CONFLICT, "Contest is no longer accepting changes");
        }
        return work(repos, fn => hooks.push(fn));
    }, options), { label: `contest ${contestId} mutation` });
    await Promise.allSettled(hooks.map(fn => fn()));   // post-commit, never fails the request
    return result;
};
```

P3. Contest lifecycle as an explicit state machine

Pattern	State (table-driven finite state machine)
Where	ContestStatus usage: contest.service.ts:48-64 (five status lists), 778-786, 937, 1677; Agenda/jobs.ts:18-42, 79-94, 125-169; finalization.service.ts:447-516, 559-607; contestLifecycle.ts
SOLID	O (new states = table edit), S (lifecycle rules in one module)
Effort / risk	3-4 days | Medium: status is everywhere. Add the module first, then migrate callers one by one

Problem today. Legal transitions are implicit and scattered. Agenda activates contests in two different jobs: `contest:checkUpcoming` uses an unguarded update and `contest:active` a guarded one. Only the second is scheduled today (Agenda/index.ts:38), but the first is a latent race. Questions like 'can it be edited?', 'is it frozen?' and 'does it show in the Ended tab?' are answered by different hand-written arrays. The enum also mixes lifecycle states (UPCOMING, ACTIVE, FINALIZING...) with view filters (JOINED, OPEN, NEW), and getContestsByStatus maps CLOSED to a COMPLETED query (1545-1549).

Why this pattern. A transition table makes every legal move explicit, testable in isolation and enforced in one place by a conditional write (the guarded pattern the code already trusts). Capability predicates replace the scattered arrays. Separating view filters from lifecycle states stops UI concerns leaking into persistence.

```
// Contest/lifecycle/contestStateMachine.ts
const transitions: Partial<Record<ContestStatus, ContestStatus[]>> = {
    UPCOMING:            ["ACTIVE"],
    ACTIVE:              ["FINALIZING"],
    FINALIZING:          ["COMPLETED", "CLOSED", "FINALIZATION_FAILED"],
    FINALIZATION_FAILED: ["FINALIZING"],
    CLOSED:              ["COMPLETED"],              // POTY: admin publishes results
};

export const contestState = {
    canEdit:            (s: ContestStatus) => s === "NEW" || s === "UPCOMING",
    acceptsEntries:     (s: ContestStatus) => s === "ACTIVE",
    isFrozen:           (s: ContestStatus) => ["FINALIZING", "COMPLETED", "CLOSED"].includes(s),
    tab:                (s: ContestStatus): "active" | "ended" =>
                            ["COMPLETED", "CLOSED", "FINALIZATION_FAILED"].includes(s) ? "ended" :
                                "active",

    // The ONLY way to change status. Atomic, idempotent, and it refuses illegal moves.
    async transition(db: PrismaTx, contestId: string, to: ContestStatus,
                     extra: Prisma.ContestUpdateManyMutationInput = {}) {
        const from = (Object.keys(transitions) as ContestStatus[])
            .filter(s => transitions[s]!.includes(to));
        const res = await db.contest.updateMany({
            where: { id: contestId, status: { in: from } },
            data: { status: to, ...extra },
        });
        return res.count === 1;
    },
};

// Agenda job becomes:
if (await contestState.transition(prisma, c.id, "ACTIVE", { startedAt: now })) {
    await agenda.schedule(c.endDate, "contest:watcher", { contestId: c.id });
}
```

P4. Contest-type policies instead of isPhotographyOfYear branches

Pattern	Strategy (with a small Factory to select the policy)
Where	contest.service.ts:381-393, 411, 769-776, 798-809, 837-850, 888, 902, 992, 1004, 2261, 2298; vote.service.ts addOneVote; finalization.service.ts:500-516; photographyOfTheYear.service.ts:21-52
SOLID	O (new types without editing flows), S, I (removes boolean flag parameters), L (all policies substitutable)
Effort / risk	4-5 days | Medium: verify POTY behaviour with characterization tests first

Problem today. POTY behaviour is spread across core flows as conditionals and as a boolean argument (`allowPhotographyOfYear`) on three public functions. POTY also forces values from outside (photographyOfTheYear.service.ts overrides prizes, coin rules and money fields before calling createContest), so the rules for one type live in two modules. A third type, such as a team contest, a sponsored contest or a judged-only contest, would multiply every branch.

Why this pattern. Strategy captures each type's variable decisions (rule normalisation, awards, upload limit, entry payment mode, whether it accepts votes, how it finalizes) behind one interface. Core flows ask the policy instead of branching, which satisfies Open/Closed: a new type is a new policy file plus a registry entry.

```
// Contest/policies/contestTypePolicy.ts
export type EntryMode = "FREE" | "COINS" | "STRIPE";

export type ContestTypePolicy = {
    kind: "REGULAR" | "POTY";
    normalizeRules(rules?: ContestRuleConfigInput[], useDefaults?: boolean): ContestRuleConfigInput[];
    resolveAwards(input: { prizeIds?: string[]; prizes?: AwardInput[] }): Promise<AwardRow[]>;
    sanitizeCommercials(input: CommercialInput): CommercialInput;  // money/coin fields
    entryMode(contest: Pick<Contest, "entryFeeAmount" | "entryFeeCoins">): EntryMode;
    acceptsDirectVotes: boolean;
    finalize(ctx: FinalizationContext): Promise<void>;
};

export const regularPolicy: ContestTypePolicy = {
    kind: "REGULAR",
    normalizeRules: (r, d = r === undefined) => contestRuleService.normalizeContestRules(r, d),
    resolveAwards: ({ prizeIds = [], prizes = [] }) =>
        prizeService.resolveAwardRows(prizeIds, prizes, prizeIds.length === 0 && prizes.length === 0),
    sanitizeCommercials: c => c,
    entryMode: c => c.entryFeeAmount > 0 ? "STRIPE" : c.entryFeeCoins > 0 ? "COINS" : "FREE",
    acceptsDirectVotes: true,
    finalize: regularFinalization,                    // ranking + grants + payouts
};

export const potyPolicy: ContestTypePolicy = {
    kind: "POTY",
    normalizeRules: r => contestRuleService.normalizeContestRules(r)
        .map(rule => rule.key === "SUBMISSION_LIMIT" ? { ...rule, value: 8, enabled: true } : rule),
    resolveAwards: async () => [],
    sanitizeCommercials: c => ({ ...c, isMoneyContest: false, minPrize: 0, maxPrize: 0,
        entryFeeCoins: 0 }),
    entryMode: () => "STRIPE",
    acceptsDirectVotes: false,
    finalize: potyFinalization,                       // close and wait for judging
};

export const policyFor = (c: { isPhotographyOfYear?: boolean | null }) =>
    c.isPhotographyOfYear ? potyPolicy : regularPolicy;

// Before:  if (contest.entryFeeAmount > 0 || allowPhotographyOfYear) throw PAYMENT_REQUIRED
// After:   if (policyFor(contest).entryMode(contest) === "STRIPE") throw PAYMENT_REQUIRED
```

P5. One command runner for paid in-contest actions

Pattern	Command + Template Method (built on P2)
Where	promoteContestPhoto 2529-2598; tradePhoto 2658-2789; chargePhoto 2855-2892; chargeContestEntryFee 203-230
SOLID	S, O (new actions = new command), D (runner depends on repo ports)
Effort / risk	3-4 days | Low-Medium: fixes R1 as a side effect; cover with concurrency tests

Problem today. Promote, trade and charge each re-implement 'check the contest, check ownership, check the balance, spend one item, mutate, invalidate caches, schedule follow-ups'. They do it differently: promote checks the balance *outside* the transaction and decrements unconditionally inside it, which is a real overspend race (R1). Trade reads and then writes inside the transaction. Only the entry fee uses the safe conditional decrement (`updateMany where coins >= fee`).

Why this pattern. Modelling each action as a Command object (validate, apply, afterCommit) with one runner guarantees every action spends through the same atomic, conditional decrement and the same guard. It also gives one natural place for audit logging, metrics, rate limiting and a future store ledger. New items, such as a 'spotlight' or 'extend' action, become new commands.

```
// Contest/actions/consume.ts - the ONLY way to spend a store item
export type StoreItem = "boost" | "swap" | "key" | "coins";

export const consume = async (db: PrismaTx, userId: string, item: StoreItem, amount = 1) => {
    const res = await db.userStore.updateMany({
        where: { userId, [item]: { gte: amount } },
        data:  { [item]: { decrement: amount } },
    });
    if (res.count !== 1) throw new ApiError(httpstatus.PAYMENT_REQUIRED, `Not enough
        ${itemLabel[item]}`);
};

// Contest/actions/contestAction.ts
export type ContestAction<I extends { userId: string; contestId: string }, O> = {
    name: string;
    cost?: { item: StoreItem; amount: number };
    precheck?(input: I): Promise<void>;                 // cheap, outside the transaction
    apply(repos: ContestRepos, input: I): Promise<O>;    // inside the transaction
    afterCommit?(input: I, output: O): Promise<unknown>;
};

export const runContestAction = async <I extends { userId: string; contestId: string }, O>(
    action: ContestAction<I, O>, input: I,
) => {
    await action.precheck?.(input);
    return withActiveContest(input.contestId, async (repos, afterCommit) => {
        if (action.cost) await repos.store.consume(input.userId, action.cost.item,
            action.cost.amount);
        const out = await action.apply(repos, input);
        afterCommit(() => bus.publish("contest.store.spent", { ...input, action: action.name }));
        if (action.afterCommit) afterCommit(() => action.afterCommit!(input, out));
        return out;
    });
};

export const promoteAction: ContestAction<PromoteInput, { promotionExpiresAt: Date }> = {
    name: "promote",
    cost: { item: "boost", amount: 1 },
    async apply(repos, { contestId, slotId, userId }) {
        const slot = await repos.slots.findLiveForOwner(slotId, contestId, userId);
        if (!slot) throw new ApiError(httpstatus.CONFLICT, "Contest photo is no longer eligible for
            promotion");
        if (isPromotionActive(slot)) throw new ApiError(httpstatus.BAD_REQUEST, "Contest photo is
            already promoted.");
        const promotionExpiresAt = new Date(Date.now() + PROMOTION_DURATION_MS);
        await repos.slots.markPromoted(slot.id, promotionExpiresAt);
        return { promotionExpiresAt };
    },
    afterCommit: (i, o) => agenda.schedule(o.promotionExpiresAt, "promotion:remove", { photoId:
        i.slotId }),
};
```

P6. Rule engine as a handler registry with one rule-set load

Pattern	Registry + Strategy per rule key + Chain of Responsibility over apply points
Where	ContestRules/contestRule.definitions.ts:193-251 (field switch); contestRule.engine.ts:88-167, 303-313, 330-389; contestRules.service.ts:116-200; contest.service.ts:1281-1342 (duplicate formatter)
SOLID	O (new rule = new handler), S (one file per rule), D (handlers receive context, not globals)
Effort / risk	4-5 days | Low-Medium: existing validation tests (contest.validation.test.ts, submissionFormat.test.ts) are a good safety net

Problem today. Each rule's behaviour is split across a definition entry, an admin-fields switch, a validator inside the engine, and a summary formatter that exists twice (contest.service.ts and contestRules.service.ts). Every `getEnabledRuleValue` call reloads all rule configs from the database: the join path loads them 3 times and the upload path several more times per request (R7). The `appliesTo` metadata ('JOIN', 'PHOTO_UPLOAD', 'VOTING') is declared but not used to drive execution.

Why this pattern. A registry keyed by rule key, where each handler owns its schema, admin fields, summary and hook functions, means a new rule is one new file. Chain of Responsibility, driven by the existing `appliesTo` metadata, runs exactly the handlers for the current apply point. Loading a `RuleSet` once per request and passing it down removes the repeated queries.

```
// ContestRules/handlers/types.ts
export type RuleContext = { contestId: string; userId: string; rules: RuleSet; tx?: PrismaTx };
export type RuleHandler<V> = {
    key: ContestRuleKey;
    schema: z.ZodType<V>;
    fields: ContestRuleFieldDefinition[];
    summarize(value: V): string;
    onJoin?(ctx: RuleContext & { accepted: Set<ContestRuleKey> }, value: V): Promise<void> | void;
    onUpload?(ctx: RuleContext & UploadInput, value: V): Promise<void> | void;
    onVote?(ctx: RuleContext & VoteInput, value: V): Promise<void> | void;
};

// ContestRules/handlers/submissionLimit.handler.ts
export const submissionLimitHandler: RuleHandler<number> = {
    key: "SUBMISSION_LIMIT",
    schema: z.number().int().min(1).max(50),
    fields: [{ path: "value", label: "Submission limit", inputType: "number", required: true }],
    summarize: v => `${v} photo submits per participant`,
    async onUpload(ctx, limit) {
        const existing = await countLiveSlots(ctx.contestId, ctx.participantId);
        if (existing + ctx.incomingCount > limit) throw new ApiError(400, "Maximum upload limit
            exceeded");
    },
};

// ContestRules/ruleRegistry.ts
export const ruleHandlers: Record<ContestRuleKey, RuleHandler<any>> = {
    SUBMISSION_LIMIT: submissionLimitHandler, SUBMISSION_FORMAT: submissionFormatHandler,
    SUBMISSION_RULES: submissionRulesHandler, LEVEL_REQUIREMENTS: levelRequirementsHandler,
    ELIGIBILITY: eligibilityHandler, COPYRIGHT: copyrightHandler,
    VOTING: votingHandler, PARTICIPATION: participationHandler,
};

// One DB read per request, then a chain over the enabled rules for this apply point.
export const runRules = async (point: "onJoin" | "onUpload" | "onVote", ctx: Omit<RuleContext,
    "rules">, input: object) => {
    const rules = await loadRuleSet(ctx.contestId);
    for (const { key, value } of rules.enabled()) {
        await ruleHandlers[key][point]?.({ ...ctx, ...input, rules } as any, value);
    }
};
```

P7. A single vote-tally function

Pattern	Strategy (scoring policy) / single source of truth, as a pure function
Where	ContestRanking/contestRanking.service.ts:164-199; Vote/vote.service.ts getVoteCount and getVoteCountsForContestPhotos; contest.service.ts:1603-1608 (getContestUploadsByUserId), 1789-1793, 2977-2981 (dead)
SOLID	S, O (versioned policies), D (pure core, I/O at the edges)
Effort / risk	2-3 days | Low for the function; product sign-off needed on which number the UI should show

Problem today. The slot score is *weighted live votes in the current stint + bankedVotes + initialVotes (original photo only)*. That formula is re-implemented in four places with slightly different terms. getVoteCountsForContestPhotos returns live + banked with no initialVotes, and getContestUploadsByUserId then *subtracts* initialVotes from that result, so 'My contest' counts can differ from the leaderboard (R3). `CONTEST_SCORING_VERSION` exists, but nothing ties the read paths to it.

Why this pattern. A pure, unit-tested `tallySlot()` becomes the one formula. The ranking scan, bulk counts and single counts all feed it votes. Treating it as a versioned Strategy (keyed by scoringVersion) lets you change scoring for new contests later without silently re-scoring finished ones.

```
// Contest/scoring/voteTally.ts  (pure, no I/O, 100% unit-testable)
export type SlotState = { id: string; photoId: string | null; originalPhotoId: string | null;
                          initialVotes: number | null; bankedVotes: number | null;
                          stintStartedAt: Date | null; createdAt: Date };
export type VoteRow   = { photoRefId: string | null; createdAt: Date; power: number | null };

export const countsForStint = (slot: SlotState, vote: VoteRow) =>
    (vote.photoRefId === null || vote.photoRefId === slot.photoId) &&
    vote.createdAt >= (slot.stintStartedAt ?? slot.createdAt);

export const scoringPolicies = {
    2: (slot: SlotState, votes: VoteRow[]) => {
        const live = votes.filter(v => countsForStint(slot, v));
        const initial = !slot.originalPhotoId || slot.originalPhotoId === slot.photoId ?
            slot.initialVotes ?? 0 : 0;
        return {
            score: live.reduce((n, v) => n + getVoteWeight(v), 0) + (slot.bankedVotes ?? 0) + initial,
            voterCount: live.length,
        };
    },
} as const;

export const tallySlot = (slot: SlotState, votes: VoteRow[], version: keyof typeof scoringPolicies =
    2) =>
    scoringPolicies[version](slot, votes);
```

P8. Domain events + transactional outbox for side effects

Pattern	Observer (typed in-process bus) + Transactional Outbox for must-not-lose events
Where	contest.service.ts:93-105 (dynamic Team import), 1030-1034, 1105-1111, 2340-2352, 2585-2594, 2783-2785; vote.service.ts runVoteSideEffects / handleVoteSideEffects; Agenda/jobs.ts:140-141, 163-164; event/eventEmitter.ts (unused)
SOLID	D (publishers do not know subscribers), O (new reactions without touching writers), S
Effort / risk	6-8 days (bus 1 day; outbox + dispatcher 3 days; migrate call sites) | Medium: change delivery semantics deliberately, one event type at a time

Problem today. Writers call every interested module directly: the user-store cache, profile cache, ranking memo, team matchmaking, POTY top-ten, level service and notifications. That is why Contest imports Team and Team imports Contest. Some effects are fire-and-forget promises, and a crash between commit and effect silently drops them (for example the team-queue advance or the promotion-expiry schedule). Vote side effects use runInBackground, which is in-memory only.

Why this pattern. With Observer, a writer only announces what happened, and modules subscribe to what they care about. That removes the import cycles and makes it easy to add features (analytics, webhooks, achievements). For effects that must not be lost (team-queue advance, payouts, POTY notifications), the Outbox writes the event in the *same transaction* as the change, and an Agenda job dispatches it with retries. That gives at-least-once delivery, and the existing idempotency-key discipline makes handlers safe to repeat.

```
// shared/domainEvents.ts
export type DomainEvents = {
    "contest.joined":        { contestId: string; userId: string; via: EntryMode | "UPLOAD" };
    "contest.photo.changed": { contestId: string; userId: string; reason: "SUBMIT" | "TRADE" |
        "WITHDRAW" | "MODERATE" };
    "contest.vote.cast":     { contestId: string; voterId: string; slotIds: string[] };
    "contest.finalized":     { contestId: string };
    "contest.store.spent":   { contestId: string; userId: string; action: string };
};

const handlers = new Map<keyof DomainEvents, Array<(p: any) => Promise<unknown>>>();
export const bus = {
    on<K extends keyof DomainEvents>(k: K, h: (p: DomainEvents[K]) => Promise<unknown>) {
        handlers.set(k, [...(handlers.get(k) ?? []), h]);
    },
    async publish<K extends keyof DomainEvents>(k: K, p: DomainEvents[K]) {
        const res = await Promise.allSettled((handlers.get(k) ?? []).map(h => h(p)));
        res.forEach(r => r.status === "rejected" && logger.error({ err: r.reason, event: k }, "Event
            handler failed"));
    },
    // Outbox: durable, written inside the caller's transaction, dispatched by Agenda.
    record: <K extends keyof DomainEvents>(tx: PrismaTx, k: K, p: DomainEvents[K]) =>
        tx.domainEventOutbox.create({ data: { type: k, payload: p as any, status: "PENDING" } }),
};

// Subscribers live in their own modules - Contest no longer imports Team:
// Team/team.subscribers.ts
bus.on("contest.joined", ({ userId, contestId }) => teamQueue.onMemberJoinedContest(userId,
    contestId));
bus.on("contest.finalized", ({ contestId }) => teamService.closeActiveMatchesForContest(contestId));
// Contest/contest.subscribers.ts
bus.on("contest.photo.changed", async ({ contestId, userId }) => {
    contestRankingService.invalidateContestRanking(contestId);
    await profileCache.invalidateUser(userId);
});
```

P9. Contest blueprint factory (replace the unused builders)

Pattern	Factory Method + Builder (typed, immutable result)
Where	contest.service.ts:362-450 (createContest), 463-655 (materializeRecurringOccurrence), 657-753 (createRecurringContest), 756-925 (updateContest); classes/*
SOLID	S, O, L (a typed result replaces the any-bag), DRY
Effort / risk	3-4 days | Low-Medium: covered by create/materialize characterization tests

Problem today. Creating a contest row with its rule configs, awards and level awards is written three times. The award column mapping is copied by hand (580-597), so a new Prize column must be added in several places or it will be silently dropped from recurring instances. Banner resolution is copied three times. The existing Builder classes aimed at this but are unused and incorrect (R8).

Why this pattern. A factory with one constructor per source (admin request, recurring template) produces a typed `ContestBlueprint`, and one `persistBlueprint` writes it. The duplication collapses, the ordering of writes is fixed in one place, and the factories are pure enough to unit-test. A Builder is still useful inside the factory for the optional parts (money settings, banner), as long as it is typed and validates correctly.

```
// Contest/authoring/contestBlueprint.ts
export type ContestBlueprint = {
    contest: Prisma.ContestUncheckedCreateInput;
    rules: ContestRuleConfigInput[];
    awards: AwardRow[];
    levelAwards: LevelAwardRow[];
};
```

```
export const contestBlueprints = {
    async fromAdminRequest(creatorId: string, body: contestData, banner: ResolvedBanner):
        Promise<ContestBlueprint> {
        const policy = policyFor(body);
        const rules = policy.normalizeRules(body.rules, body.rules === undefined);
        return {
            contest: {
                id: banner.recordId, creatorId, title: body.title, description: body.description,
                status: "UPCOMING", startDate: new Date(body.startDate), endDate: new Date(
                    body.endDate),
                isPhotographyOfYear: policy.kind === "POTY",
                ...policy.sanitizeCommercials(commercialsFrom(body)),
                ...(await contestCategoryService.prepareCategoryUpdate(resolveCategoryInput(body))),
                maxUpload: contestRuleService.getSubmissionLimitFromRules(rules),
                ...banner.fields,
            },
            rules,
            awards: await policy.resolveAwards(body),
            levelAwards: policy.kind === "POTY" ? [] : body.levelAwards ?? [],
        };
    },
    async fromRecurringTemplate(t: RecurringContest, at: Date): Promise<ContestBlueprint> { /* ... */
        },
};

export const persistBlueprint = async (tx: PrismaTx, bp: ContestBlueprint) => {
    const contest = await tx.contest.create({ data: bp.contest });
    await insertRuleConfigs(tx, contest.id, bp.rules);         // one implementation
    await insertAwards(tx, contest.id, bp.awards);             // one column mapping
    await insertLevelAwards(tx, contest.id, bp.levelAwards);
    return contest;
};
```

P10. Banner and storage behind an adapter

Pattern	Adapter + small Strategy (source of banner)
Where	contest.service.ts:189-201, 372-378, 705-714, 817-832, 915-917; helpers/fileUploader; Uploader/classes/*
SOLID	D, S
Effort / risk	1-2 days | Low

Problem today. The three banner branches (pick an existing user photo, upload a new file, or neither) each handle 'discard the uploaded file if a user photo was chosen' and 'delete the replaced file after commit'. Business code calls the S3/DigitalOcean helper directly.

Why this pattern. A `bannerResolver.resolve()` returns the fields to persist plus a cleanup callback. The storage provider sits behind the existing `IProvider` adapter, so tests use an in-memory store and a provider change does not touch contest code.

```
export const bannerResolver = {
    async resolve(input: { userPhotoId?: string; file?: Express.Multer.File }, storage: StoragePort) {
        if (input.userPhotoId) {
            const fromPhoto = await resolveBannerFromUserPhoto(input.userPhotoId);
            if (input.file) await storage.discard(input.file);
            return { fields: { banner: fromPhoto!.banner, bannerUploaderId:
                fromPhoto!.bannerUploaderId } };
        }
        if (input.file) {
            const key = (await storage.upload(input.file)).Key;
            return { fields: { banner: key, bannerUploaderId: null }, rollback: () => storage.discard(
                input.file!) };
        }
        return { fields: {} };
    },
};
```

P11. Caching as a decorator around read loaders

Pattern	Decorator (higher-order function) / Proxy
Where	contest.service.ts:1124-1145 (detail), 1423-1510 (list), 1798-1833 (completed card), 1839-1847 (winners); ContestRanking memo
SOLID	S, O
Effort / risk	1 day | Low

Problem today. contest.cache.ts is well designed, but business functions call it inline and must know the part names, TTLs and scopes. Read-model loaders and caching are interleaved, which makes both harder to test.

Why this pattern. Wrapping a pure loader with a caching decorator keeps the loader testable without Redis, and keeps cache policy (part, TTL, scope) declared in one place, next to the invalidation event subscriptions.

Multi-instance note. The ranking memo in contestRanking.service.ts is per process. Production does run several instances (ecosystem.config.js: cluster, instances: max), so `invalidateContestRanking` only clears the local copy. The 3-second TTL bounds the staleness, and finalization correctly bypasses the memo (maxAgeMs: 0). Measurement shows the bigger issue is rebuild cost, not staleness: see PF3.

```
// Contest/read/cachedLoaders.ts
export const cachedPerContest = <T>(part: ContestCachePart, load: (c: CacheableContest) => Promise<T>,
                                    opts?: CacheOptions) =>
    (contest: CacheableContest) => contestCache.getOne(part, contest, load, opts);

export const loadDetailExtras = cachedPerContest("detail", rawLoadDetailExtras);
export const loadWinners      = cachedPerContest("winners", c => rawLoadWinners(c.id));
```

P12. Contest list queries as specifications

Pattern	Specification / Query Object
Where	getAllContests 1196-1236; getPublicContests 1238-1279; getContestsByStatus 1513-1575; getMyCompletedContest 1798-1833; notDeleted copies (Section 2)
SOLID	S, O, DRY
Effort / risk	2 days | Low

Problem today. Each list endpoint hand-builds `where`, `orderBy`, the include, count + findMany, pagination meta and enrichment. Tie-break ordering (always ending on id) is a correctness rule that must be remembered in every copy.

Why this pattern. A specification names each audience's view of contests ('admin list', 'public by status', 'discover for user') and returns only `{ where, orderBy }`. One runner handles pagination, includes, stable ordering and enrichment. Shared scopes (`notDeleted`, `activeContestWhere`) live in one module.

```
// Contest/read/contestScopes.ts
export const scopes = {
    notDeleted: { OR: [{ deletedAt: null }, { deletedAt: { isSet: false } }] } satisfies
        Prisma.ContestWhereInput,
    regularOnly: { isPhotographyOfYear: false },
};

type ListSpec = { where: Prisma.ContestWhereInput; orderBy: Prisma.ContestOrderByWithRelationInput[]
    };

export const contestListSpecs = {
    admin: (o: { status?: ContestStatus; tab?: ContestTab; search?: string; includeArchived?: boolean
        }): ListSpec => ({
        where: { ...scopes.regularOnly, ...statusFilter(o), ...titleSearch(o.search),
                 ...(o.includeArchived ? {} : scopes.notDeleted) },
        orderBy: [{ startDate: "desc" }, { id: "desc" }],   // every spec ends on id: stable pages
    }),
    discoverForUser: (userId: string, status: ContestStatus): ListSpec => ({ /* ACTIVE: endDate asc
        ... */ }),
};
```

```
export const runContestList = async (spec: ListSpec, page = 1, limit = 20) => {
    const p = paginationHelper.calculatePagination({ page, limit });
    const [rows, total] = await Promise.all([
        prisma.contest.findMany({ where: spec.where, orderBy: spec.orderBy, skip: p.skip, take:
            p.limit, include: contestCardInclude }),
        prisma.contest.count({ where: spec.where }),
    ]);
    return paginated(await enrichContestListDetails(rows), total, p);
};
```

P13. Presenters / mappers for response shapes

Pattern	Presenter (Data Mapper for read models) + versioned DTOs
Where	contest.service.ts:1596-1624 (five aliases for one number), 1792-1795, 2037-2050, 2080-2096, 2120-2141, 3024-3043, 3121-3161; v2/ (empty)
SOLID	S, O (new API version = new presenter)
Effort / risk	2-3 days | Low

Problem today. Response shaping is embedded in queries. One photo returns `voteCount`, `totalVotes`, `votes`, `vote_count` and `total_votes` for client compatibility. Similar uploads views exist in near-identical copies. The empty v2 folder suggests a new API version was planned, but there is no seam to support one.

Why this pattern. Presenters separate 'what the domain knows' from 'what each client version receives'. v1 presenters keep the legacy aliases, and v2 presenters can be clean. Domain functions then return one shape, which makes them simpler and testable.

```
// Contest/presenters/contestPhoto.presenter.ts
export const contestPhotoPresenter = {
    ownerV1: (slot: SlotView, tally: Tally) => ({
        ...slot, userPhotoId: slot.photo.id, url: slot.photo.url,
        voteCount: tally.score, totalVotes: tally.score, votes: tally.score,   // legacy aliases kept
            here only
        vote_count: tally.score, total_votes: tally.score, traded: slot.traded,
    }),
    ownerV2: (slot: SlotView, tally: Tally) => ({
        id: slot.id, photo: { id: slot.photo.id, url: slot.photo.url },
        votes: { score: tally.score, voters: tally.voterCount }, traded: slot.traded,
    }),
};
```

P14. Thin controllers with validated request DTOs

Pattern	Adapter (HTTP to use case) + DTO validation (existing validateRequest middleware)
Where	contest.controller.ts (all handlers); contest.route.ts (only 4 of 34 routes use validateRequest; duplicate prize-selections / award-selections routes 31-45)
SOLID	S, I, D
Effort / risk	2-3 days | Low

Problem today. Handlers parse query strings by hand (`Number(page) || 1`), cast `req: any` and pass loosely typed values into services that then defend themselves (for example getContestsByStatus re-validates the status enum). Validation is inconsistent across endpoints.

Why this pattern. zod schemas for *params, query and body* on every route, handled by the existing middleware, give services typed, trusted input. A tiny handler adapter removes the repeated sendResponse boilerplate. Services then stop throwing HTTP-shaped validation errors for malformed input.

```
// contest.validation.ts
export const listContestsQuery = z.object({ query: z.object({
    page: z.coerce.number().int().min(1).default(1),
    limit: z.coerce.number().int().min(1).max(100).default(20),
    status: z.nativeEnum(ContestStatus).optional(),
    tab: z.enum(["active", "ended"]).optional(),
    search: z.string().trim().max(100).optional(),
    includeArchived: z.coerce.boolean().default(false),
```

```
}) });

// shared/handler.ts
export const handle = <T>(message: string, fn: (req: Request) => Promise<T>, status = 200) =>
    catchAsync(async (req, res) => sendResponse(res, { statusCode: status, success: true, message,
        data: await fn(req) }));

// contest.route.ts
router.get("/all", auth(UserRole.ADMIN), validateRequest(listContestsQuery),
    handle("All contests fetched successfully", req => contestQuery.adminList(req.query as
        ListContestsQuery)));
```

6. Reliability findings

These were found while tracing the code for the pattern analysis. Items marked **Verify** depend on intended product behaviour and should be confirmed before changing. The pattern that removes the root cause is listed for each item, so fixing it is part of the refactor and does not need a separate patch. Failed votes under concurrency are also a reliability problem; see PF1 in Section 7.

#	Severity	Finding and location	Fix / pattern
R1	High	**Promote can overspend boosts.** The balance is read outside the transaction (contest.service.ts:2554-2558), and inside it the decrement is unconditional (2574-2577) with no balance re-check. Two concurrent promotes with boost = 1 can both succeed, leaving boost = -1.	Conditional decrement via `consume()` (P5). Quick fix: `updateMany where boost >= 1`, check count
R2	High	**Moderation removal races finalization.** adminDeleteContestPhoto checks the status (1676-1679), then updates the slot outside any transaction or guard (1681-1684). If finalization freezes the contest in between, a photo can vanish from a ranking that is being snapshotted.	Guarded update: `updateMany where contest status not frozen` inside a transaction (P2/P3)
R3	Medium	**Verify: vote counts disagree between screens.** getVoteCountsForContestPhotos returns live + banked (no initialVotes). getContestUploadsByUserId then subtracts initialVotes (1605-1608), while the leaderboard adds them (ranking.service.ts:184-194). For seeded slots, 'My contest' can show up to 2 x initialVotes less than the leaderboard.	Single tally function (P7)
R4	Medium	**Verify: reward labels look swapped.** Finalization notifications describe keyReward as 'promotes' and boostReward as 'charges' (finalization.service.ts:91-92), but promote spends `boost` (2574-2577) and charge spends `key` (2879-2882).	One `StoreItem` vocabulary with labels (P5)
R5	Medium	**Unescaped HTML in moderation e-mail.** The admin-entered `reason` and the user's fullName/username are interpolated raw into HTML (1697-1699).	Mailer adapter with escaping/templates; route through notificationOrchestrator
R6	Medium	**getParticipantLevelData throws a plain Error** (2497-2499), which becomes HTTP 500. It is called per contest inside getMyActiveContests and loadCompletedContestCard, so one inconsistent row fails the whole list.	ApiError NOT_FOUND, or return null and let the presenter degrade
R7	Medium	**Repeated rule loads per request.** Each getEnabledRuleValue call re-queries all rule configs (contestRules.service.ts:159-168). validateJoinRules makes 3 calls; validateUploadRules makes 3 or more, plus join, plus the submission limit again at contest.service.ts:2338.	RuleSet loaded once per request (P6)
R8	Medium	**Latent hazards in dead code.** replaceContestPhoto / replaceContestPhotoWithUserPhoto (2817-2852) are unexported but would bypass the active guard, swap charge, trade ledger and stint reset if wired up. The `contest:checkUpcoming` job (jobs.ts:18-42) uses an unguarded status update. The builders throw on valid dates (AbstractContestBuilder.ts:34).	Delete (Phase 0)
R9	Medium	**Failed finalization retries forever with no limit.** contest:watchEnded selects every FINALIZATION_FAILED contest on each 30-second run (Agenda/jobs.ts:145-169), and claimFinalization accepts FAILED with no cap or delay (finalization.service.ts:130-145). attemptCount is incremented but never read. A contest that always fails, for example because of bad data, repeats the work and logs the same error every run, and nobody is alerted, so its results stay unpublished until someone notices. Depending on where it fails, each retry can include a full vote scan and a ranking snapshot. Contests are processed one at a time, so a slow failing contest also delays the others in that run.	Cap attempts in the claim (attemptCount < max), back off using updatedAt, alert admins at the cap, and add a manual retry that resets the count
R10	Low	**N+1 in my-active contests.** For each contest, getMyActiveContests calls getParticipantLevelData (participant, contest, rule set, ranking) and getContestUploadsByUserId (1751-1756). getClosedContestsWithWinner does the same (1893-1901, currently unused).	Batch loaders in the read service (P1/P13); see PF11

#	Severity	Finding and location	Fix / pattern
R11	Low	**Exposure decay is row-by-row** every minute across all active participants (2894-2930).	Compute decay lazily at read time from exposureUpdatedAt, or batch by bucket
R12	Low	**The voting feed reads the whole eligible set per page** (1993-2003). This is intentional for stable seeded pagination, but cost is O(photos) per page request.	Cache the shuffled id list per (contest, voter, seed) for a few minutes
R13	Low	**Per-process ranking memo** (see P11 note). Staleness is bounded by the 3 s TTL, but production runs one process per CPU and each vote clears the memo, so it saves little under load (PF3).	Stop per-vote invalidation; share through Redis (PF3)

Unused code to delete before refactoring

Removing these shrinks the surface area that must be characterised and migrated:

- contest.service.ts: `identifyContestTopPhoto` (2646-2655), `getCompletedContestUploads` (2064-2097), `getContestPhotoVoteScore` (2977-2981), `replaceContestPhoto` + `replaceContestPhotoWithUserPhoto` (2817-2852), and the large commented-out blocks (318-359, 1944-1958, 2413-2464, 2600-2616).
- Exported but with no external callers: `getUpcomingContest`, `getClosedContestsWithWinner`. Confirm no client uses them before removing.
- classes/* builders and factory (replaced by P9), Agenda `contest:checkUpcoming` and the commented `contest:recurring`.
- Empty files: v2/contest.routes.ts, v2/contest.controller.ts, ContestPhoto/contestPhoto.service.ts, contest.enum.ts, contest.controller.spec.ts. Recreate them when they have content.

7. Performance analysis

This section covers the whole backend, not only the Contest module. The findings come from reading the code and from running **your real service functions** against a seeded test database with the MongoDB profiler recording every operation. IDs PF1 to PF14 are ordered by impact.

How this was measured

Item	Detail
Code under test	The real addOneVote, buildContestRanking, levelService, getMyActiveContests, getContestUploadsToVote, dashboard and notification services, unmodified, from this working tree
Database	Throwaway local MongoDB 7.0.14 single-node replica set. Production and your .env database were not touched: the run verified it was on the local database before doing anything, and all non-local network connections were blocked
Data	1,000,000 votes (200,000 in the contest under test), 1,000,000 notifications, 88,600 contest photos, 42,800 participants, 201 contests, 20,000 users, 100,000 payments. Indexes copied from prisma/*.prisma
Measured with	Wall-clock time plus system.profile (operations, documents examined, query plans)

How far to trust these numbers. The data is synthetic and the size is an assumption, so check your real size first with `db.votes.estimatedDocumentCount()`. Costs marked as scans grow roughly in step with collection size, so with 100,000 votes expect about a tenth of the times shown. Three things differ from production: (1) Redis was off, so application caches were bypassed and read times are cold-cache times; (2) the database was on the same machine, so there is no network delay, and on Atlas every operation costs more and transactions stay open longer; (3) only one Node process ran, while production runs one per CPU. The comparisons and failure modes are reliable; treat the exact seconds as illustrative.

Results at a glance

Operation	Today	With fix	Gain	Fix tested
Votes at 4 / 16 / 64 at once (real addOneVote)	75% / 93% / 98% fail	0% fail	Failures removed	Same transaction with a read guard (PF1)
One vote: background work	139 s, 127 DB ops, 4.29M docs	Not run end to end	-	Parts below (PF2, PF3, PF4)
Level stats for one user	145.9 s	28 ms	about 5,200x	Two-step query, same totals verified (PF2)
Contest ranking build	22.6 s	3.6 s	6x	One index (PF3)
My active contests	25.1 s	5.3 s	4.7x	Two indexes (PF6, PF11)
Voting feed, per page	2.4 s	165 ms	14x	Two-step query, data step only (PF7)
Notification list	890 ms	21 ms	42x	Two indexes (PF6)
Admin user list, 20 users	over 400 s (stopped)	Not run	-	Same cause as PF2 (PF9)
Dashboard overview	4.8 s, 26 ops	Not run whole	-	Income step 162 ms with groupBy (PF9)

Fast paths confirmed: the vote response itself took 18-32 ms, and the admin contest list took 548 ms. The upload pipeline streams to storage instead of holding files in memory, and direct-to-storage upload is already supported.

PF1. Votes fail when several arrive at once (Critical)

Where. vote.service.ts addOneVote and addVotes (the guard at the start of each transaction); error.middleware.ts:87-97.

What happens. Each vote transaction writes `updatedAt` on the same contest document as a guard. In MongoDB, if two transactions write the same document, the second is aborted immediately instead of waiting. So all votes in one contest compete for one document. Prisma raises P2034, addOneVote only handles P2002 and does not retry, and the error middleware returns **HTTP 400 'The database could not complete this request'**. Promote, trade and upload use the same guard, but they are far less frequent.

Votes at once	Today: real addOneVote (200 votes)	Current + retry (300)	Read guard (300)
1	200 ok, 0 failed	300 ok, 142/s	300 ok, 150/s
4	**50 ok, 150 failed (75%)**	300 ok, 104/s, p95 15 ms	300 ok, 508/s, p95 11 ms
16	**14 ok, 186 failed (93%)**	300 ok, 93/s, p95 1.1 s	300 ok, 1,240/s, p95 17 ms
64	**4 ok, 196 failed (98%)**	274 ok, 26 failed, 76/s, p95 2.1 s	300 ok, 1,310/s, p95 60 ms

The columns are from separate runs of the same load pattern: all votes target one contest and each client votes again as soon as the last vote returns. Real traffic is less tightly packed, but only two transactions need to overlap to cause a failure, and on Atlas each transaction stays open longer than the 6-8 ms measured here.

Why retry is not the full answer. Retry stops most failures, so it is a good same-day patch, but votes still run one at a time and p95 reaches 1-2 seconds. Removing the shared write gives roughly 13-17x more throughput at 16-64 votes at once.

Permanent fix. Votes only need to know the contest is open, which a read answers. The write guard exists so the finalizer cannot miss a late vote. Because endDate is fixed once a contest is ACTIVE (updateContest rejects it), time can do that job instead: count only votes before endDate, and start finalizing after a grace period longer than the 15-second vote transaction timeout. Keep the write guard for trades, promotes and uploads. This changes a finalization guarantee, so add a test for a vote committing right at endDate.

```
// vote.service.ts - inside the transaction: read, do not write the contest document
const open = await tx.contest.findFirst({
    where: { id: contestId, ...activeContestWhere() }, select: { id: true },
});
if (!open) throw new ApiError(httpstatus.CONFLICT, "Contest voting has closed");

// contestFinalization.service.ts - wait until every in-flight vote has finished
const FREEZE_GRACE_MS = 20_000;     // must stay above the vote transaction timeout (15 s)
if (contest.endDate.getTime() + FREEZE_GRACE_MS > Date.now()) return null;  // the 30 s sweep retries

// contestRanking.service.ts - the ranking ignores anything after the deadline
where: { contestId, createdAt: { lt: contestEndDate } }

// Same-day patch until then: replay on conflict
const result = await runWithWriteConflictRetry(() => prisma.$transaction(/* ... */), { label: "vote"
    });
```

PF2. 'Votes received by a user' scans every vote in the database (Critical)

Where. level.service.ts:131-146 (runs after every vote), and vote.service.ts getUserTotalVotes, getTotalPromotedVotes and getTotalOrganicVotes. These feed profile stats, Team getMembers (once per member) and the admin user list (once per user).

Why it is slow. The filter `{ photo: { participant: { userId } } }` crosses two relations. The profiler shows Prisma turning it into an aggregation that starts from **every vote** and runs a `$lookup` into contest_photos and then contest_participants for each one, filtering only at the end. No index can help, because the user is only known after both joins: adding indexes left it at 152 s. One owner with 128 votes took **145.9 s** and examined 4 million documents.

Fix. Walk the relation from the user side instead. Each step is indexed. This ran in **28 ms** and returned identical totals (verified: 128 votes either way). It needs the indexes from PF3 and PF6. Long term, keep a counter:

`User.totalVotes` already exists in the schema, but nothing writes to it. Note that a counter must follow the same trade and stint rules as the ranking.

```
const getReceivedVoteStats = async (userId: string) => {
    const participants = await prisma.contestParticipant.findMany({ where: { userId }, select: { id:
        true } });
    const slots = await prisma.contestPhoto.findMany({
        where: { participantId: { in: participants.map(p => p.id) } }, select: { id: true },
    });
    const where = { contestPhotoId: { in: slots.map(s => s.id) } };
    const [all, promoted] = await Promise.all([
        getVoteWeightStats(where),
        getVoteWeightStats({ ...where, type: VoteType.Promoted }),
    ]);
    return { receivedVotes: all.weight, receivedVoteCount: all.count,
             promotedVotes: promoted.weight, promotedVoteCount: promoted.count };
};
```

PF3. The ranking is rebuilt constantly, and one index is missing (Critical)

Where. contestRanking.service.ts:119-247; vote.service.ts handleVoteSideEffects; prisma/contest.prisma (ContestPhoto).

Why it is slow. A ranking loads each participant's photos by `participantId`, but the only matching index starts with contestId. MongoDB therefore scans all 88,600 contest photos, which took 19.8 of the 22.6 seconds. Adding one index brought it to **3.6 s**.

Why it runs so often. The 3-second memo is meant to stop repeat builds, but every vote calls invalidateContestRanking first, and the side effects then rebuild it. During steady voting the memo rarely helps, and each worker keeps scanning. The memo is also per process, and production runs one process per CPU (ecosystem.config.js: cluster, instances: max), so invalidation on one worker does not reach the others. These rankings also feed vote-count polling and **three public routes with no login or rate limit** (contest.route.ts:48-50).

Fix, in order. (1) Add the index. (2) Stop invalidating on every vote, and let the short TTL do the work so staleness stays within seconds; finalization already bypasses the memo. (3) Move the shared ranking to Redis so all workers reuse one build. (4) Let the database total votes: `vote.groupBy` took 571 ms for the vote step. It needs a small extra step for traded photos, so it is not a drop-in replacement. (5) For very large contests, keep live scores in a Redis sorted set updated on each vote and rebuild from MongoDB only at finalization.

PF4. One vote starts 127 database operations in the background (Critical)

Where. vote.service.ts runVoteSideEffects (lines 53-79); backgroundTasks.ts; photographyOfTheYearRanking.service.ts:177-190 (new on this branch).

The vote response is fast only because this work happens after it. The cost is still paid, once per vote:

```
POST /votes -> response in 18-32 ms, then in the background:
  evaluateParticipantLevel    -> full ranking rebuild (PF3)
  evaluateAndUpdateUserLevel  -> two scans of all votes (PF2), about 14 badge counts,
                                 user.update, userLevel.upsert even when nothing changed
  getVoteCount + notification -> 1 new notifications row per vote, 1 socket push
  POTY top-10 check           -> full POTY ranking on EVERY vote in ANY contest,
                                 about 25 more ops and 1.2 s with only 1,000 entries; no memo

Measured for one vote, POTY off: 139 s, 127 ops, 4.29M documents examined, 18 collection scans
```

Why this is dangerous. runInBackground has no limit or queue. If votes arrive faster than this work finishes, tasks pile up in memory and compete for the same database connections. This was observed directly: with background work on, a sequence of 60 votes did not finish within 120 seconds, while the same request path alone took seconds. The work is also lost on a restart, because shutdown waits at most 8 seconds.

Fix. Batch this work. Record what changed, and let a scheduled job process each participant, owner and contest once, however many votes arrived. Run the POTY check as a job every minute instead of on every vote, and make it use the memo. Only write the user and level records when a value actually changed. Group vote notifications (for example 'You received 12 votes') so one vote does not create one row. Fix the POTY check before merging this branch.

```
// After the vote commits: remember the work, do not do it now
await redisClient.sAdd("vote:dirty:owners", ownerUserIds);
await redisClient.sAdd("vote:dirty:contests", contestId);

// Agenda, every 30 seconds: each owner is processed once per run
agenda.define("votes:applyDerived", async () => {
    const owners = await redisClient.sPop("vote:dirty:owners", 200);
    for (const ownerId of owners) await levelService.evaluateAndUpdateUserLevel(ownerId);
});
agenda.define("poty:checkTopTen", () =>
    photographyOfTheYearRankingService.notifyTopTenForActiveContests());
```

PF5. Real-time events do not cross workers (High)

Where. helpers/websocketSetUp.ts:19 and 38 (adapter commented out); ecosystem.config.js. **Found by reading the configuration; not measured.**

Why it matters. Production runs in PM2 cluster mode, but the Socket.IO Redis adapter is commented out. A notification sent from worker A only reaches sockets connected to worker A, so a user on another worker misses it until they refresh. Cluster mode also needs sticky sessions for Socket.IO's HTTP polling transport. `getOnlineCount()` reports one worker's connections only. Each worker's startup sets `isOnline` to false for all users, including those connected to other workers. `io.emit('user_status')` also sends every sign-in to every socket.

Fix. Enable the adapter (redisSubClient is already prepared for it) after connectRedis, set up sticky sessions or use WebSocket-only transport, and send status changes only to followers or teammates.

PF6. Missing indexes (High)

MongoDB scans a whole collection when no index matches. Each of these is one schema line. Build them during quiet hours, because indexing large collections adds load.

Add to prisma schema	Query it serves	Measured
ContestPhoto `@@index([participantId])`	Ranking, level stats, my contests	22.6 s to 3.6 s (ranking)
ContestParticipant `@@index([userId, status])`	My contests, 'playing now', achievements	With the line above: 25.1 s to 5.3 s
Notification `@@index([receiverId, createdAt(sort: Desc)])` and `@@index([receiverId, isRead])`	Notification list and unread count. The collection has no indexes today, and it gains a row per vote	890 ms to 21 ms
Payment `@@index([status, type, createdAt])`	Dashboard and year filters	Not measured separately
Comment `@@index([parentId])`	Reply tree (comment.service.ts:102-118)	Not measured
Follow `@@index([followingId])`	Follower lists and counts; the unique index starts with followerId	Not measured
ContestAchievement `@@index([participantId, category])`	Badge counts; about 14 per level check. Rewrite the relation filter as in PF2	Not measured (test table was empty)

PF7. The voting feed is slow for every page (High)

Where. contest.service.ts:1982-2003. **Measured:** 2.4 s per page, 218,538 documents examined.

Why. `votes: { none: { providerId } }` makes MongoDB check each photo's votes, and the participant filter adds another join. The whole eligible set is reloaded for every page, and this is the screen voters use most.

Fix. First fetch the photos this voter already voted on, using the existing index, then exclude them: this data step took 165 ms (14x). The filtered participants need the same treatment. Then cache the shuffled ID list per voter and seed for a few minutes, so page 2 costs almost nothing.

PF8. Every authenticated request reloads the full user (High)

Where. auth.middleware.ts:38-42. **Found by reading the code.** Each request runs findUnique for the whole user document. authCache exists, but only /auth/me uses it. This is cheap one at a time, but it is the busiest query in the system, and it adds one network round trip to every request on Atlas.

Fix. Select only the fields the middleware needs, and cache them in Redis for 30-60 seconds. The invalidation calls already exist on block, delete and update paths, so route them to this key.

PF9. Admin dashboard loads whole tables into memory (High)

Where. dashboard.service.ts. **Measured:** overview 4.8 s (26 ops, 18 collection scans); transaction stats 2.5 s (11 scans); user list over 400 s for 20 users (30 million documents, stopped).

Why. Overview runs its queries one after another, and calcIncomeData and getRevenueByType each load every matching payment into Node just to add up amounts. getAllUsers repeats the PF2 query per user. Nothing here is cached.

Fix. Use `payment.groupBy` (162 ms for 100,000 payments), run queries with Promise.all, and cache the dashboard for 60 seconds. getAllUsers should use one batched vote total for the page. It also has a separate bug: it skips `(page - 1) * 10` whatever the limit is (line 370).

Other findings (Medium)

ID	Finding	Why it matters	Fix
PF10	**Exposure decay** (contest.service.ts:2894-2930). Measured: 12.3 s and 4,602 ops for about 2,300 participants	Runs every minute and updates rows one at a time; these writes can also collide with vote transactions	Calculate decay when reading, from exposureUpdatedAt, and store it only on change
PF11	**N+1 reads.** getMyActiveContests 1751-1756; getContestsByStatus completed cards (2.1 s); Team getMembers; Team contest list 2412-2414; Profile getPhotos 182-183	Cost grows with each item on the page, and each item runs several queries	Load in one batch by IDs, as loadContestListExtras already does well
PF12	**No image sizes and no response compression.** No resize library, and no compression middleware in app.ts	Feeds appear to serve originals of up to 25 MB, the largest cost for mobile users. JSON lists are sent uncompressed unless a proxy handles it	Generate thumbnails at upload or use an image CDN; enable compression at the proxy or in Express. Confirm the proxy setup first
PF13	**Process settings.** instances: max with a 4 GB heap each; no pool size in DATABASE_URL; Agenda runs in every worker; processEvery 30 s with a 5 s job; no rate limiting	Heap limits can exceed server RAM; worker count multiplied by pool size must stay under the Atlas connection limit. Check whether contest activation can lag up to 30 s	Size heap to RAM, set maxPoolSize deliberately, run jobs in one dedicated worker, rate limit public ranking routes
PF14	**Sequential fan-out.** Team payouts and team notifications await one user at a time (team.service.ts:1807-1852; 4 loops in notificationOrchestrator.ts)	Large teams make jobs slow, and each step is a separate write	Batch writes with createMany and send socket pushes to the team room

Performance action plan

Step	Do	Effort	Result
1. This week	Add runWithWriteConflictRetry to both vote functions. Map P2034 to 503 instead of 400. Add the PF6 indexes. Switch getReceivedVoteStats and getUserTotalVotes to the two-step query. Move the POTY check to a job before merging this branch	2-3 days	Votes stop failing in most cases; the largest scans disappear
2. Next	Read guard plus time-based freeze (PF1). Batch post-vote work (PF4). Stop per-vote ranking invalidation and share rankings through Redis (PF3). Enable the Socket.IO adapter (PF5)	1-2 weeks	Vote throughput limit removed; background load bounded
3. Then	Voting feed (PF7), auth cache (PF8), dashboard (PF9), N+1 batching (PF11), lazy decay (PF10), rate limits	1-2 weeks	Faster screens and lower database load

Step	Do	Effort	Result
4. Later	Thumbnails and compression (PF12), process sizing (PF13), Redis sorted-set leaderboards if needed	1-2 weeks	Lower bandwidth; room for larger contests

Keep it measured. Add the benchmark scenarios above to a repeatable load test (for example k6 against staging), turn on the Atlas Performance Advisor and slow query log, and set targets such as: no failed votes at expected peak, background vote work finished within 30 seconds, and p95 under 300 ms for the main screens. Step 1 should be released on its own, then re-measured before starting step 2.

8. Implementation roadmap

Each phase ships independently and keeps the facade and HTTP contract stable. Dependencies flow downwards: later phases build on the primitives from earlier ones. Step 1 of the performance plan (Section 7) should ship before Phase 0, because votes are failing now.

Phase	Work	Patterns	Estimate	Exit criteria
0. Safety net	Characterization tests for create, update, join (free/coins/stripe), upload (gallery + file + rollback), trade, promote, charge, finalize (regular + POTY), materialize. Fix R1, R2, R5, R6, R9. Delete unused code. Confirm R3/R4 with product.	-	5-7 days	Tests green on current behaviour; defects fixed
1. Primitives	contestScopes; contestState machine; withActiveContest + repos; consume(); tallySlot(); claimLease() helper	State, Template Method, Repository/UoW, Strategy	6-8 days	No hand-written status arrays or guards remain in new code; all counts use tallySlot
2. Decompose	Split contest.service.ts into the modules in P1; facade re-exports; Agenda/Team/POTY import narrow modules	Facade, SRP	4-6 days	contest.service.ts < 100 LOC, facade only
3. Extensibility	Contest-type policies; rule-handler registry + RuleSet; blueprint factory; banner adapter	Strategy, Registry, CoR, Factory, Adapter	10-12 days	isPhotographyOfYear only read by policyFor(); a new rule touches 1 file
4. Decoupling	Typed domain events; subscribers per module; outbox collection + Agenda dispatcher; remove the dynamic Team import	Observer, Outbox	6-8 days	No Contest to Team import; side effects survive a crash
5. Edges	Specifications for lists; presenters (v1 aliases, v2 clean); zod DTOs on all routes; caching decorators	Specification, Presenter, Decorator, Adapter	6-8 days	Controllers < 10 lines each; v2 routes possible

Total: roughly 7-9 developer-weeks spread over 2-3 months alongside feature work. Phases 0-2 alone (about 3-4 weeks) deliver most of the maintainability gain and remove the high-severity risks.

Testing strategy that makes this safe

- **Characterization first.** Record what the current code returns for each use case before moving it. The node:test runner is already set up (npm run test:contest).
- **Pure cores get unit tests.** tallySlot, contestState, the policies, rule handlers, blueprint factories and presenters are pure or take repos as parameters, so they can be tested without MongoDB.
- **Repository fakes for use cases.** Because repos are created from a tx parameter (P2), use-case tests pass an in-memory implementation.
- **Concurrency tests for money paths.** Fire N parallel promote/trade/charge/join requests against a test replica set and assert that balances never go negative and ledgers stay unique.
- **Contract tests for HTTP.** Snapshot the JSON shapes of the top endpoints so presenter changes cannot break mobile or web clients unnoticed.

Target module layout

```
src/app/modules/Contest/
  contest.service.ts            # facade only (re-exports)        <- P1
  contest.route.ts / .controller.ts / .validation.ts             <- P14
  lifecycle/contestStateMachine.ts, contestScopes.ts             <- P3, P12
  policies/contestTypePolicy.ts, regular.policy.ts, poty.policy.ts <- P4
  authoring/contestAuthoring.service.ts, contestBlueprint.ts, bannerResolver.ts <- P9, P10
  recurring/recurringMaterializer.ts
  entry/contestEntry.service.ts                                   # join, paid join, submit, withdraw
  actions/consume.ts, contestAction.ts, promote|trade|charge.action.ts <- P5
```

```
scoring/voteTally.ts                                          <- P7
read/contestQuery.service.ts, contestLeaderboard.service.ts, cachedLoaders.ts <- P11, P12
presenters/*.presenter.ts                                     <- P13
moderation/contestModeration.service.ts
persistence/contestRepos.ts, unitOfWork.ts                    <- P2
events/contest.events.ts, contest.subscribers.ts              <- P8
ContestRules/ handlers/*.handler.ts, ruleRegistry.ts, ruleSet.ts <- P6
ContestRanking/  ContestFinalization/  (kept; finalization uses policies + state machine)
```

9. Guardrails: when not to apply a pattern

- **Do not introduce class hierarchies for their own sake.** The codebase is functional and object-literal based. Strategies, commands and repositories work well as typed objects of functions, and switching style mid-codebase would hurt readability.
- **Do not abstract single implementations prematurely.** Payment has one provider. Keep the Registry, but do not build an Abstract Factory on top of it.
- **Do not put the outbox on every event.** Use the in-process bus for cache invalidation (losing one only costs a TTL). Reserve the outbox for effects that change state or money, or that users must see.
- **Do not replace the working reliability mechanisms.** Leases, idempotency keys, the versioned cache and guarded writes are correct. Generalise them into helpers; do not redesign them.
- **Do not change scoring semantics during the refactor.** Keep tallySlot behaviour identical to computeContestRanking until product explicitly approves a change, then ship it as a new scoringVersion.

Appendix A. Pattern to location matrix

Pattern	Primary locations	Rec.
Facade	contest.service.ts (whole), external importers	P1
Repository + Unit of Work	19 $transaction blocks; structural tx types 66-81	P2
Template Method	Active-contest guard x5; post-commit hooks	P2, P5
State	ContestStatus arrays 48-64, 778-786; Agenda jobs; finalization transitions	P3
Strategy (contest type)	28 isPhotographyOfYear branches; POTY service overrides	P4
Command	promote 2529, trade 2658, charge 2855, entry fee 203	P5
Registry + Chain of Responsibility	Rule engine, definitions, duplicated formatter	P6
Strategy (scoring, versioned)	Ranking scan, voteService counts, uploads views	P7
Observer + Outbox	Invalidation sequences, team queue, vote side effects, Agenda finalization follow-ups	P8
Factory + Builder	createContest / createRecurringContest / materialize; classes/*	P9
Adapter	Banner/storage, e-mail, HTTP controllers	P10, P14
Decorator / Proxy	contestCache call sites, ranking memo	P11
Specification / Query Object	Three list endpoints; notDeleted copies	P12
Presenter / Data Mapper	Response shaping and legacy aliases	P13

Line numbers refer to the working tree on branch *poty* at the time of analysis (including uncommitted changes) and will drift as the code changes. Effort figures are planning estimates, not commitments.